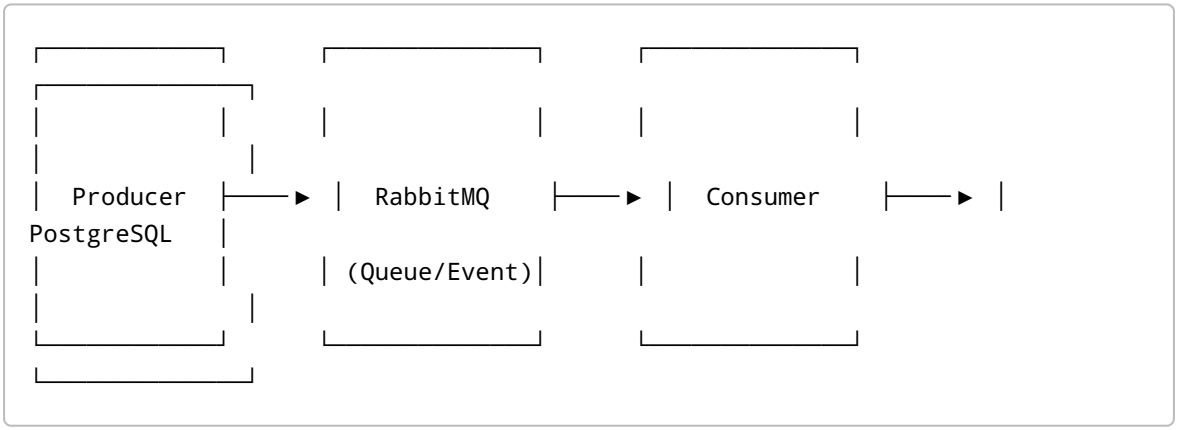
CX Data Platform

📌Visão Geral

A **CX Data Platform** é uma plataforma de ingestão e processamento de eventos orientada a mensagens, construída com foco em **boas práticas de arquitetura de dados**, **mensageria assíncrona** e **escalabilidade**.

O projeto simula um cenário real de mercado onde múltiplos produtores enviam eventos para um broker (RabbitMQ), consumidores processam esses eventos e persistem os dados em um banco relacional (PostgreSQL), preparando o terreno para análises futuras.

🧱Arquitetura Geral



Fluxo de dados

1. **Producer** gera eventos (JSON)
2. Eventos são enviados para uma **fila RabbitMQ**
3. **Consumer** consome, valida e processa o evento
4. Dados válidos são persistidos no **PostgreSQL**

📂Estrutura do Projeto

```
cx-data-platform/
├── .venv/              # Ambiente virtual Python
├── architecture/       # Diagramas e decisões arquiteturais
├── consumers/          # Serviços consumidores de eventos
│   ├── __init__.py
│   └── consumer.py
├── docker/             # Infraestrutura (Docker Compose)
│   └── docker-compose.yml
├── governance/         # Políticas e governança de dados
```

```
├── ingestion/          # Ingestão de dados externos (batch / APIs)
├── processing/         # Processamento e transformação de dados
├── producers/          # Serviços produtores de eventos
│   └── producer.py
├── schemas/            # Schemas e contratos de eventos
│   └── event_schema.py
├── storage/            # Camada de acesso e consultas
└── README.md
```

🔌Tecnologias Utilizadas

- **Python 3.13**
- **RabbitMQ** (mensageria)
- **PostgreSQL 15** (persistência)
- **Docker & Docker Compose**
- **pika** (cliente RabbitMQ)
- **psycopg2** (cliente PostgreSQL)

📐Contrato de Eventos (Schema)

Todos os eventos devem seguir um formato padronizado para garantir integridade e compatibilidade:

```
{
  "event_type": "string",
  "user_id": 123,
  "metadata": { "any": "value" },
  "created_at": "ISO-8601 datetime"
}
```

Eventos fora desse padrão são rejeitados pelo consumer.

📠Infraestrutura

Subir os serviços necessários:

```
docker compose up -d
```

Serviços disponíveis: - RabbitMQ: http://localhost:15672 - PostgreSQL: localhost:5432

📩Execução Local

Producer

```
python producers/producer.py
```

Consumer

```
python consumers/consumer.py
```

🕐Status Atual do Projeto

✔️Infraestrutura funcionando ✔️Mensageria validada ✔️Persistência validada ✔️Estrutura escalável definida

---

🕐Roadmap (Próximos Passos)

- Dead Letter Queue (DLQ)
- Versionamento de eventos
- Logs estruturados
- Métricas de consumo
- Camada de processamento
- Integração com BI / Analytics

Autor

Projeto desenvolvido por **Paulo Roberto Apolinário da Silva**

> Este projeto tem fins educacionais e arquiteturais, simulando cenários reais de plataformas de dados modernas.